



MZ-UCA Cloud Academy

Terraform Engineering Bootcamp

Student Handbook — Week 1

Terraform Foundations

Infrastructure as Code (IaC)

Instructor: Mthulisi Zulu
Cloud Governance Architect
Founder | MZ-UCA Cloud Academy

Welcome to Terraform Engineering

Welcome to the beginning of your Terraform Engineering journey.

Modern organizations no longer build infrastructure manually. Enterprise environments are created, managed, secured, and automated using code.

Terraform is one of the world's leading *Infrastructure as Code (IaC)* platforms, enabling engineers to define infrastructure using configuration files and deploy consistently across multiple technology platforms.

During this bootcamp, you will learn how to think like a modern Cloud Engineer.

You are not learning only commands.

You are learning an engineering discipline.

Week 1 Learning Objectives

By the end of this module, you should understand:

- ✓ What Infrastructure is
- ✓ What Infrastructure as Code means

- ✓ Why organizations use Terraform
 - ✓ Declarative vs Imperative approaches
 - ✓ Terraform architecture
 - ✓ Providers
 - ✓ Resources
 - ✓ State management
 - ✓ Terraform workflow
 - ✓ Basic Terraform commands
 - ✓ How to create your first Terraform-managed resources
-

1. What is Infrastructure?

Definition

Infrastructure is the collection of technology components required to run applications and deliver IT services.

Infrastructure includes:

Compute

Examples:

- Physical servers
 - Virtual machines
 - Containers
 - Serverless platforms
-

Networking

Examples:

- IP addresses
- Virtual networks

- Routers
 - Firewalls
 - Load balancers
-

Storage

Examples:

- File storage
 - Object storage
 - Block storage
-

Databases

Examples:

- SQL databases
 - NoSQL databases
 - Data warehouses
-

Security

Examples:

- Identity management
 - Access control
 - Encryption
 - Monitoring
-

Real-World Example

When you open a banking application, you only see the application.

Behind the application exists:

Mobile App

↓

Application Servers

↓

Database

↓

Network

↓

Security Controls

↓

Storage Systems

All these components together form infrastructure.

2. Traditional Infrastructure

Before cloud automation, engineers created infrastructure manually.

A typical process:

Request Server

↓

Create Machine

↓

Install Operating System

↓

Configure Network

↓

Install Application

↓

Configure Security

↓

Release to Users

Problems With Manual Infrastructure

1. Human Error

Different engineers may configure systems differently.

Example:

Server A:

```
Java version 17
```

Server B:

```
Java version 11
```

This creates inconsistency.

2. Slow Deployment

Creating infrastructure manually takes time.

Large environments may require days or weeks.

3. Poor Reproducibility

If a server is deleted, recreating the exact environment becomes difficult.

4. Difficult Scaling

Modern companies need hundreds or thousands of resources quickly.

Manual processes cannot scale.

3. Infrastructure as Code (IaC)

Definition

Infrastructure as Code is the practice of managing and provisioning infrastructure using code instead of manual processes.

Instead of clicking through dashboards:

Engineers write configuration.

Example:

```
resource "server" "application" {  
  
}
```

The code becomes the instruction for creating infrastructure.

Benefits of Infrastructure as Code

Automation

Infrastructure can be created automatically.

Consistency

The same code produces the same result.

Version Control

Infrastructure changes can be tracked using Git.

Collaboration

Teams can review infrastructure changes.

Recovery

Infrastructure can be recreated from code.

Governance

Organizations can enforce standards.

4. Why Terraform?

Terraform is an Infrastructure as Code tool created by HashiCorp.

It allows engineers to define and manage infrastructure using configuration files.

Terraform Advantages

Multi-Platform Support

Terraform works with:

- ☁ AWS
 - ☁ Microsoft Azure
 - ☁ Google Cloud Platform
 - ☁ Kubernetes
 - ☁ Docker
 - ☁ VMware
 - ☁ Oracle Cloud
 - ☁ Many other platforms
-

Terraform Philosophy

Terraform follows this principle:

Define the desired state, and Terraform will determine how to achieve it.

5. Declarative vs Imperative

Imperative Approach

You explain:

"How should this happen?"

Example:

Making tea:

1. Boil water
2. Add tea
3. Add milk
4. Add sugar
5. Stir

You provide every step.

Declarative Approach

You explain:

"What result do you want?"

Example:

"I want a cup of tea."

The system determines the required steps.

Terraform Uses Declarative Configuration

You describe:

"I need this infrastructure."

Terraform determines:

- What exists
 - What is missing
 - What needs changing
-

6. Terraform Architecture

Terraform works through several components.

Engineer

↓

Terraform Configuration

↓

Terraform CLI

↓

Provider

↓

Platform API

↓

Infrastructure

Terraform CLI

The Terraform command-line tool.

Examples:

```
terraform plan
```

```
terraform apply
```

Provider

A provider is a plugin that allows Terraform to communicate with a specific platform.

Examples:

Provider	Manages
AWS	AWS resources
Azure	Azure resources
Docker	Containers
Kubernetes	Kubernetes objects
Local	Local computer resources

Resource

A resource is something Terraform creates or manages.

Examples:

Cloud:

- Virtual machine
- Database
- Network

Local:

- File
- Folder

Example:

```
resource "local_file" "example" {  
}
```

State File

Terraform requires memory.

That memory is stored in:

```
terraform.tfstate
```

Why State Exists

Terraform needs to know:

- What resources it created
- Current configuration
- Previous changes

Without state:

Terraform would not know what already exists.

Important Concept

The Terraform state file is Terraform's memory.

7. Terraform Workflow

Professional Terraform workflow:

Write Configuration

↓

Initialize Project

↓

Validate Code

↓

Create Plan

↓

Apply Changes

↓

Manage Infrastructure

8. Terraform Commands

terraform init

Purpose

Initialize a Terraform project.

Command:

```
terraform init
```

Terraform will:

- Download providers
- Prepare the project
- Create Terraform working files

Use when:

- Starting a new project
 - Adding providers
-

terraform validate

Purpose

Check whether Terraform configuration is correct.

Command:

```
terraform validate
```

Checks:

- Syntax
 - Configuration errors
-

terraform fmt

Purpose

Automatically format Terraform code.

Command:

```
terraform fmt
```

Why?

Professional teams maintain clean and consistent code.

terraform plan

Purpose

Preview changes before applying.

Command:

```
terraform plan
```

Terraform compares:

Desired State

against

Current State

Then creates an execution plan.

terraform apply

Purpose

Create or modify infrastructure.

Command:

```
terraform apply
```

Terraform:

1. Executes the plan
2. Creates resources
3. Updates state

terraform destroy

Purpose

Remove resources managed by Terraform.

Command:

```
terraform destroy
```

Important:

Always use carefully in production environments.

9. First Terraform Project

Project structure:

```
terraform-foundations/  
main.tf  
terraform.tfstate  
.terraform/
```

Local Provider

For this course, we begin with the Local Provider.

Why?

Because:

- No cloud account required
 - No billing required
 - Students can practice locally
 - Focus remains on Terraform concepts
-

Lab 1

Create welcome.txt

Terraform will create:

```
welcome.txt
```

Content:

```
Welcome to Terraform Engineering
```

```
MZ-UCA Cloud Academy
```

Workflow:

```
terraform init
```

```
terraform validate
```

```
terraform fmt
```

```
terraform plan
```

```
terraform apply
```

Lab 2

Generate README.md

Terraform creates documentation automatically.

Example:

```
Terraform Engineering Bootcamp
```

```
Student:
```

```
Course:
```

```
Date:
```

Lab 3

Generate company-policy.txt

Terraform creates a company policy document.

Example:

Password Policy

Data Protection Policy

Acceptable Usage Policy

Homework Assignment

Create a Terraform project that generates:

employees.txt

departments.txt

README.md

company-policy.txt

cloud-engineering-notes.txt

Requirements:

- ✓ Use Terraform
 - ✓ Use Local Provider
 - ✓ Store project on GitHub
 - ✓ Include README documentation
-

Week 1 Summary

You learned:

- ✓ Infrastructure concepts
- ✓ Traditional infrastructure challenges
- ✓ Infrastructure as Code
- ✓ Terraform fundamentals
- ✓ Terraform architecture
- ✓ Providers

- ✓ Resources
 - ✓ State management
 - ✓ Terraform workflow
 - ✓ Terraform commands
-

Interview Questions

1. What is Infrastructure as Code?

Answer:

Infrastructure as Code is the practice of managing infrastructure through code instead of manual configuration.

2. Why is Terraform called declarative?

Answer:

Because engineers define the desired final state, and Terraform determines the required actions.

3. What is a Terraform provider?

Answer:

A provider is a plugin that allows Terraform to communicate with a specific platform or service.

4. What is Terraform state?

Answer:

Terraform state is the file that stores information about resources managed by Terraform.

5. Difference between terraform plan and terraform apply?

Answer:

Plan previews changes. Apply executes changes.

Final Thought

"A beginner uses cloud consoles.

A Cloud Engineer builds repeatable systems through code."

The journey from clicking buttons to engineering infrastructure begins here.

 *MZ-UCA Cloud Academy*

Mthulisi Zulu

Cloud Governance Architect

Building Cloud Engineers, Not Just Certification Holders